

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– Ho Chi Minh, October 2026–

Table of Contents

I. Record of Changes..... 3
II. Software Design Document..... 4
1. System Design.....4
1.1 System Architecture.....4
1.2 Package Diagram.....6
2. Database Design.....8
3. Detailed Design.....13
3.1 <Feature/Function Name1>.....13
3.2 <Feature/Function Name2>.....15
4. State Machine Diagram..... 15
4.1. GuestVisit.....15
5. Activity Diagram.....15
5.1. Face & User Enrollment.....15
5.2. Zone Commissioning & Camera Surveillance Deployment..... 16
5.3. Zone Access Granting..... 17
5.4. <Tên workflow 4>.....18
5.5. Incident Alert Handling & Dispatch..... 18
5.6. 24/7 Video & Incident Evidence Recording.....20
6. Technology Disclosure.....20

I. Record of Changes

Date	A* M, D	In charge	Change Description
25/10/2026	A	Phan Gia Anh	Add "1.1 System Architecture"
25/10/2026	A	Phan Gia Anh	Add "1.2 Package Diagram"
27/10/2026	A	Phan Gia Anh	Add State Machine Diagram of "GuestVisit" lifecycle
28/10/2026	A	Phan Gia Anh	Add "Activity Diagram Zone Commissioning & Camera Surveillance Deployment"
28/10/2026	A	Phan Gia Anh	Add "Activity Diagram Incident Alert Handling & Dispatch"
28/10/2026	A	Phạm Nguyên Bảo	Add "Activity Diagram 24/7 Video & Incident Evidence Recording"
28/10/2026	A	Nguyễn Anh Hào	Add "Activity Diagram Face & User Enrollment"
28/10/2026	A	Nguyễn Anh Hào	Add "Activity Diagram Zone Access Granting"
28/10/2026	A	Nguyễn Hoàng Minh	Add "Database Design"
28/10/2026	A	Nguyễn Hoàng Minh	Add "Add Activity Diagram Real-time AI-powered Surveillance"

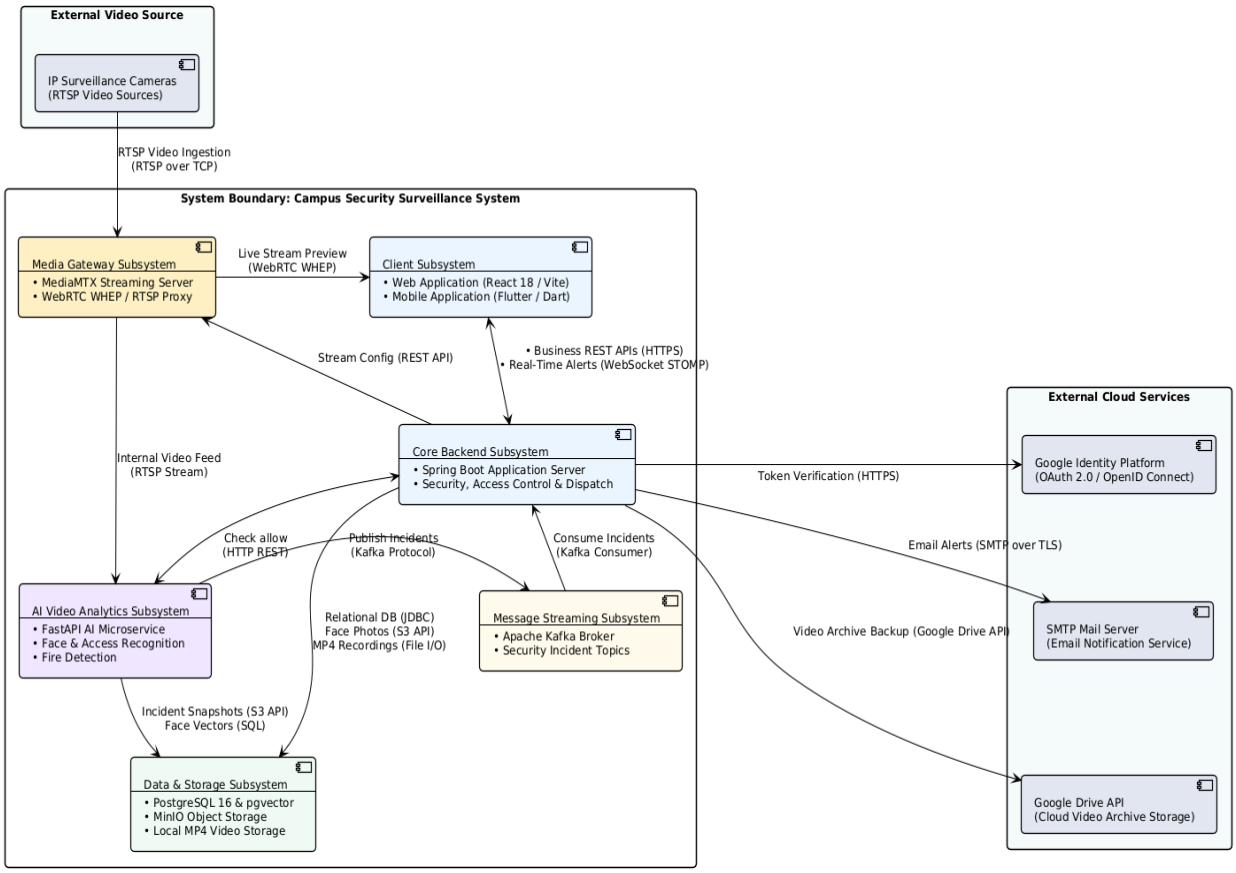
*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

Link diagram PlantUML: Ấn vào đây



The overall system architecture of the AI-Based Intelligent Campus Security Surveillance System is structured into distinct internal sub-systems enclosed within the System Boundary, alongside external sources and cloud integrations. Below is the concise explanation for each diagram component:

1. External Systems & Hardware

- IP Surveillance Cameras (RTSP Video Sources): Physical campus IP cameras that capture surveillance footage and stream continuous H.264 video feeds to the system over RTSP (TCP).
- Google Identity Platform (OAuth 2.0 / OpenID Connect): External identity provider used by the backend to authenticate campus users via institutional Google accounts securely.
- SMTP Mail Server (Email Notification Service): Mail transport service used by the core backend to deliver account credential emails, password reset tokens, and urgent system alerts.

- Google Drive API (Cloud Video Archive Storage): Remote cloud backup destination used by the backend for long-term off-site archival and scheduled backups of surveillance video recordings.

2. Internal Sub-systems (System Boundary)

- ❖ Media Gateway Subsystem (MediaMTX Streaming Server):
  - ➢ Role: Acts as the unified video ingestion and distribution hub.
  - ➢ Function: Ingests raw RTSP streams from external IP cameras, re-streams low-latency live video to client web/mobile applications via WebRTC (WHEP), and proxies internal RTSP video feeds to the AI analytics sub-system, protecting physical camera hardware from connection overload.
- ❖ Client Subsystem (Web & Mobile Applications):
  - ➢ Role: Unified user interface layer for administrators, facility managers, guards, and general campus users.
  - ➢ Function:
    - ■ Web App (React 18 / Vite): Provides live camera wall monitoring, floor plan navigation, access request approvals, guard shift scheduling, and security audit logs.
    - ■ Mobile App (Flutter / Dart): Empowers on-duty security guards to receive instant incident push alerts, claim incident handling assignments, and submit on-site resolution evidence.
- ❖ AI Video Analytics Subsystem (FastAPI Microservice):
  - ➢ Role: Computer vision and intelligent surveillance processing engine.
  - ➢ Function: Consumes real-time video frames from the Media Gateway; performs person detection, tracking, facial feature extraction, and custom anomaly detection (e.g., restricted area intrusion, unauthorized access). Interacts with the backend via HTTP REST (Check allow) to verify detected individuals' access permissions against current area rules.
- ❖ Message Streaming Subsystem (Apache Kafka Broker):
  - ➢ Role: High-throughput, asynchronous message buffering and event distribution backbone.
  - ➢ Function: Decouples the AI analytics engine from the backend business services. Receives real-time security incident events published by the AI service and reliably delivers them to backend consumers without data loss during traffic spikes.
- ❖ Core Backend Subsystem (Spring Boot Application Server):
  - ➢ Role: Central business logic, security policy orchestration, and workflow controller.

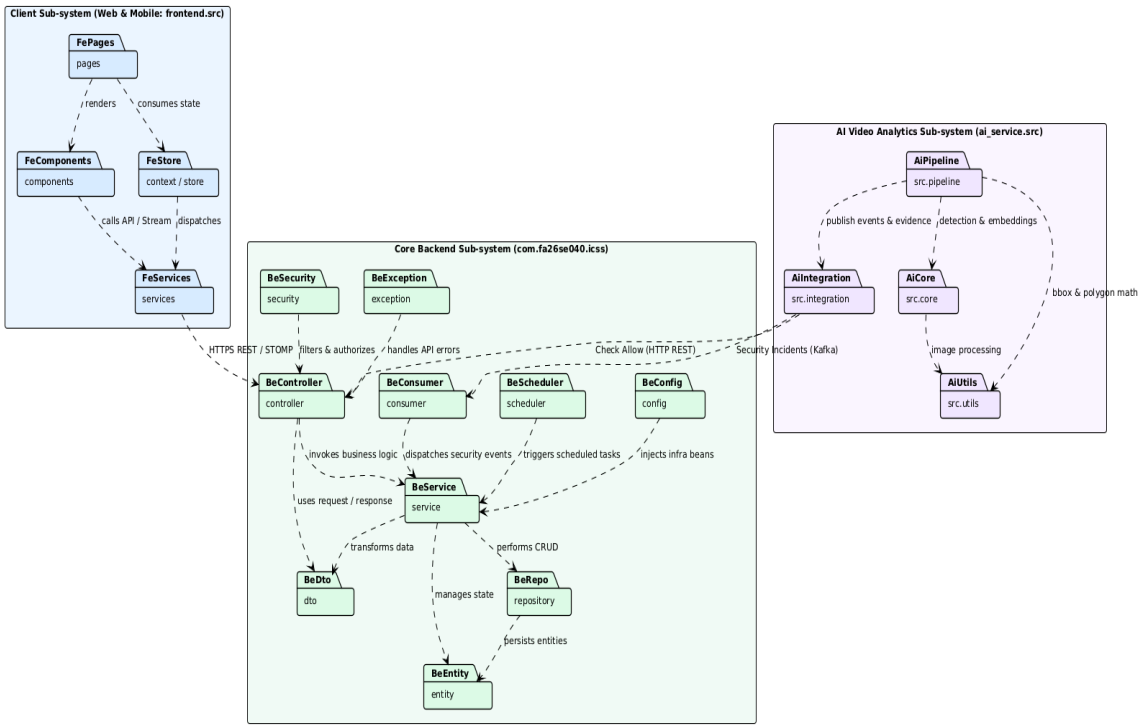
- ➢ Function: Handles RBAC security authorization, access request lifecycle management, guard staffing & shift dispatching, camera stream configuration via MediaMTX REST APIs, incident verification workflows, and real-time alert broadcasting to client applications via WebSocket (STOMP).

❖ Data & Storage Subsystem:

- ➢ Role: Comprehensive multi-modal persistence layer.
- ➢ Components & Functions:
  - ■ PostgreSQL 16 & pgvector: Stores relational business entities (users, roles, areas, shift schedules, incident records) and 512-dimensional biometric face embeddings for sub-millisecond similarity matching.
  - ■ MinIO Object Storage: Secure S3-compatible storage for facial profile images and high-resolution incident snapshot evidence.
  - ■ Local MP4 Video Storage: Dedicated server disk storage for continuous 24/7 video recording files generated by the media ingestion pipeline.

1.2 Package Diagram

Link diagram PlantUML: Ấn ở đây



Package Descriptions

No	Package	Description

01	com.fa26se040.icss.controller	Exposes RESTful API endpoints for client applications and internal services.
02	com.fa26se040.icss.service	Contains core business logic, access rules validation matrix, shift scheduling, and incident handling workflows.
03	com.fa26se040.icss.repository	Spring Data JPA data access interfaces for querying PostgreSQL database and pgvector collections.
04	com.fa26se040.icss.entity	Database domain models mapping to relational tables (Users, Cameras, Areas, Shifts, Incidents).
05	com.fa26se040.icss.dto	Data Transfer Objects for client request/response payloads, decoupled from internal entity structures.
06	com.fa26se040.icss.consumer	Apache Kafka event consumers receiving real-time security alerts streamed from the AI video analytics sub-system.
07	com.fa26se040.icss.scheduler	Scheduled cron jobs for automated shift transitions, incident escalation checks, and biometric expiration tasks.
08	com.fa26se040.icss.security	Authentication and authorization layer, implementing JWT token filters and Google OAuth2 verification.
09	com.fa26se040.icss.config	Infrastructure setup configuring Kafka brokers, MinIO S3 storage, WebSocket STOMP, and cross-origin policies.
10	com.fa26se040.icss.exception	Centralized exception handling and domain-specific error mapping for consistent API responses.
11	ai_service.src.core	Core computer vision algorithms: human detection (YOLO), face detection, feature embedding (ArcFace), and feature matching.
12	ai_service.src.pipeline	Video stream processing pipelines, multi-object tracking (ByteTrack), and ROI polygon boundary intrusion logic.
13	ai_service.src.integration	External clients for publishing Kafka security incidents, storing MinIO snapshots, and checking backend authorization.
14	ai_service.src.utils	Image transformation utilities, bounding box math, and spatial geometric helper functions.
15	frontend.src.pages	Routed view pages (Dashboard, Live Camera Grid, Access Request Approval, Incident Dispatch, Audit Logs).
16	frontend.src.components	Reusable UI components (Live Video Canvas, Floor Plan Overlays, ROI Polygon Editor, Modals).

17	frontend.src.services	HTTP Axios clients, WebSocket STOMP subscribers, and WebRTC WHEP player connection modules.
18	frontend.src.context	React application state stores managing active sessions, notifications, and real-time incident alerts

2. Database Design

Link ERD



Table Descriptions

No	Table	Description
01	buildings	Represents physical campus building structures (e.g., Building Alpha, Beta), tracking total floor counts and active status. - Primary keys: id - Foreign keys: None

02	floors	Represents individual floor levels within campus buildings (e.g., Ground Floor G, Floor 1, Floor 2). - Primary keys: id - Foreign keys: building_id (references buildings.id)
03	areas	Represents spatial security zones, rooms, and checkpoints, categorized by security levels (Public, Restricted, Secured) and clearance rules. - Primary keys: id - Foreign keys: floor_id (references floors.id)
04	area_level_presets	Defines system-wide baseline security policies, access level thresholds, and authorization requirements for each area classification tier. - Primary keys: area_level - Foreign keys: updated_by (references users.id)
05	guard_teams	Stores security guard teams, their organizational shift demand quotas (morning, afternoon, night, Sunday), and operational status. - Primary keys: id - Foreign keys: None
06	users	Stores system user accounts across all roles (Admin, Facility Manager, Guard, Normal User), including login credentials, clearance access levels, and team affiliations. - Primary keys: id - Foreign keys: team_id (references guard_teams.id)
07	face_data	Stores biometric facial reference profiles, portrait image URLs, and 512-dimensional facial embedding vectors for 1:1 user authentication. - Primary keys: id - Foreign keys: user_id (references users.id)
08	password_reset_tokens	Manages secure single-use tokens and expiration timestamps for self-service password recovery workflows. - Primary keys: id - Foreign keys: None
09	area_assigned_personnel	Manages explicit, time-bounded access authorizations assigned to specific users for entering restricted zones outside standard role permissions. - Primary keys: id - Foreign keys: area_id (references areas.id), user_id (references users.id), created_by (references users.id), revoked_by (references users.id)
10	cameras	Represents surveillance cameras deployed in areas, storing camera codes, connectivity status, and AI Region of Interest (ROI) geometry boundaries. - Primary keys: id - Foreign keys: area_id (references areas.id)
11	camera_stream_configurations	Stores network parameters, credentials, host/port, and RTSP stream paths (main and sub stream) for video ingestion. - Primary keys: id - Foreign keys: camera_id (references cameras.id)

12	camera_health_logs	Records operational health checks, streaming frame rates (FPS), network latency, and diagnostic error codes for cameras. - Primary keys: id - Foreign keys: camera_id (references cameras.id)
13	camera_access_logs	Logs checkpoint entry/exit authentication events, recording biometric identification decisions (Granted/Denied) and policy evaluation reasons. - Primary keys: id - Foreign keys: area_id (references areas.id), user_id (references users.id)
14	guard_shifts	Stores scheduled guard duty shifts, assigning guards to specific areas and duty time slots (Morning, Afternoon, Night) with check-in/out tracking. - Primary keys: id - Foreign keys: guard_id (references users.id), area_id (references areas.id)
15	guard_shift_requests	Handles self-service guard workflows for shift swapping, finding replacement guards, or requesting time off, along with manager review details. - Primary keys: id - Foreign keys: requester_id (references users.id), shift_id (references guard_shifts.id), substitute_guard_id (references users.id), target_shift_id (references guard_shifts.id), reviewed_by (references users.id)
16	guard_team_dispatches	Records temporary inter-team mobilizations and reassignments of guards during peak workloads, events, or security alerts. - Primary keys: id - Foreign keys: guard_id (references users.id), from_team_id (references guard_teams.id), to_team_id (references guard_teams.id), created_by (references users.id)
17	security_incidents	Records real-time security anomalies detected by AI computer vision (trespassing, loitering, violence, blacklisted person), handling guard response and resolution. - Primary keys: id - Foreign keys: area_id (references areas.id), camera_code (references cameras.camera_code), claimed_by (references users.id), resolved_by (references users.id)
18	incident_escalations	Tracks multi-tier automated escalation logs for unhandled security alerts, supervisor escalations, and incident closure reasons. - Primary keys: id - Foreign keys: incident_id (references security_incidents.id)
19	access_requests	Manages advance area access requests submitted by users for individuals or groups, tracking manager approvals and execution timeframes. - Primary keys: id

		- Foreign keys: area_id (references areas.id), requester_id (references users.id), reviewer_id (references users.id), cancelled_by (references users.id), finished_by (references users.id)
20	access_request_members	Maps participating group members (users) included under an approved group access request. - Primary keys: id - Foreign keys: access_request_id (references access_requests.id), user_id (references users.id)
21	area_event_schedules	Stores planned campus events, workshops, or academic schedules requiring temporary area unlock or altered access rules. - Primary keys: id - Foreign keys: area_id (references areas.id), created_by (references users.id), updated_by (references users.id), cancelled_by (references users.id)
22	area_event_sessions	Tracks live execution windows, ongoing sessions, and actual check-in timelines for scheduled events in an area. - Primary keys: id - Foreign keys: area_id (references areas.id), schedule_id (references area_event_schedules.id), started_by (references users.id), ended_by (references users.id)
23	guest_visits	Manages visitor registration records created by campus hosts, including visit purpose, validity duration, and review status. - Primary keys: id - Foreign keys: host_id (references users.id), reviewed_by (references users.id), revoked_by (references users.id)
24	guest_visit_areas	Many-to-many junction table mapping guest visits to the specific permitted areas visitors are authorized to access. - Primary keys: (visit_id, area_id) - Foreign keys: visit_id (references guest_visits.id), area_id (references areas.id)
25	guests	Stores guest demographic details, organization affiliation, portrait photos, and privacy consent records under a visit. - Primary keys: id - Foreign keys: visit_id (references guest_visits.id), consent_confirmed_by (references users.id)
26	guest_face_embeddings	Stores temporary facial biometric embeddings for approved visitors, enabling facial recognition with automated retention expiration. - Primary keys: guest_id - Foreign keys: guest_id (references guests.id)

27	notifications	Stores system alerts, task assignments, and push notifications delivered to users and guards. - Primary keys: id - Foreign keys: recipient_id (references users.id)
28	reason_catalog	Stores standardized system-wide reason codes and display labels for rejections, cancellations, and incident outcomes. - Primary keys: id - Foreign keys: None
29	system_configurations	Stores dynamic runtime configurations and thresholds (e.g., alert escalation timeouts, face similarity cutoff) without requiring code redeployment. - Primary keys: config_key - Foreign keys: updated_by (references users.id)
30	system_configuration_change_logs	Maintains an immutable audit trail of administrative configuration adjustments, recording old/new values and user rationale. - Primary keys: id - Foreign keys: config_key (references system_configurations.config_key), changed_by (references users.id)
31	audit_event_types	Defines category taxonomy and metadata descriptions for trackable system audit events. - Primary keys: id - Foreign keys: None
32	audit_module_roles	Maps module-level administrative access permissions required to review audit logs. - Primary keys: id - Foreign keys: None
33	audit_logs	Comprehensive security audit log capturing sensitive administrative and security-critical actions with correlation IDs. - Primary keys: id - Foreign keys: area_id (references areas.id), subject_user_id (references users.id), changed_by (references users.id)

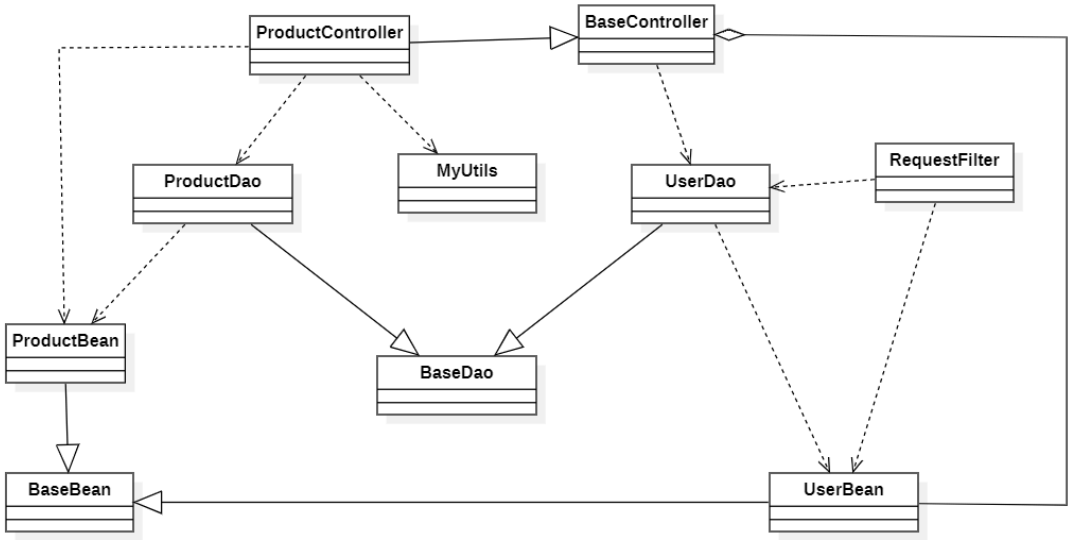
3. Detailed Design

3.1 <Feature/Function Name1>

[Provide the detailed design for the feature <Feature Name1>. It includes Class Diagram, Class Specifications, and Sequence Diagram(s); ***For the features/functions with the same structure of class & sequence diagrams, you need to provide the diagrams once for one feature/function and refer to those diagrams from other features/functions****]*

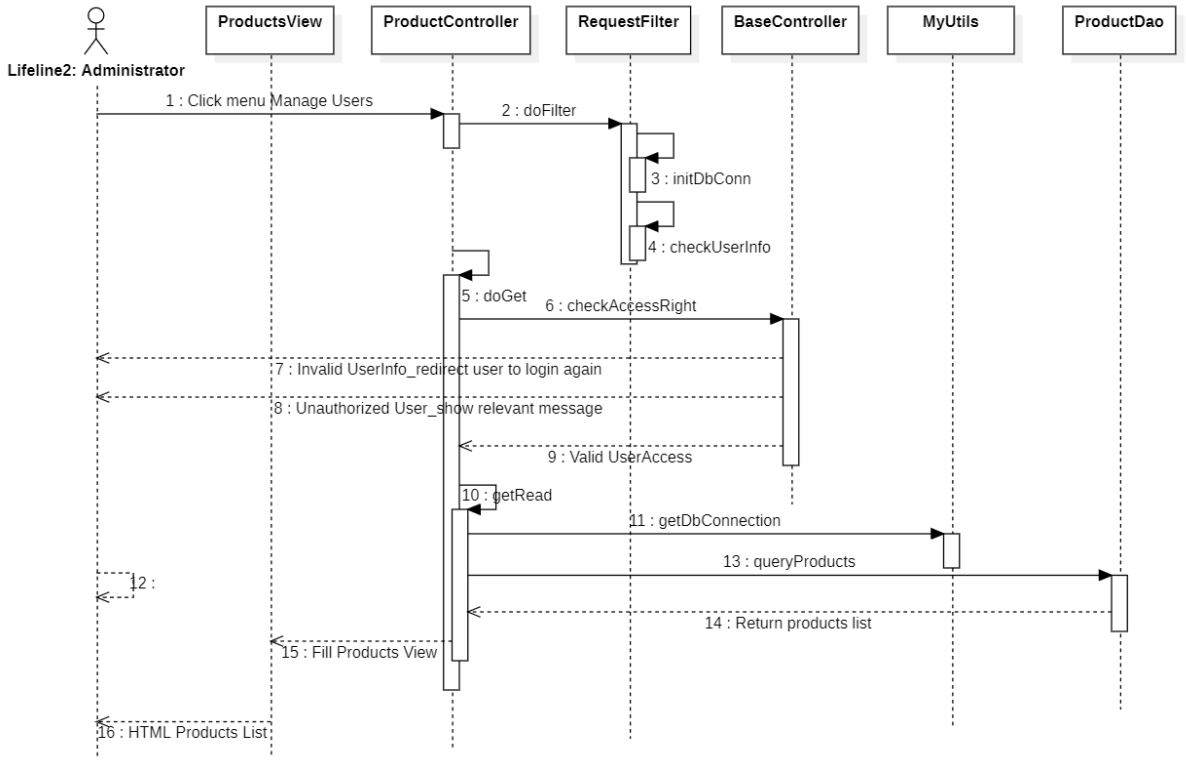
3.1.1 Class Diagram

[This part presents the class diagram for the relevant feature]



3.1.2 <Sequence Diagram Name1>

[Provide the sequence diagram(s) for the feature, see the sample below]



3.1.3 <Sequence Diagram Name2>

3.1.4 …

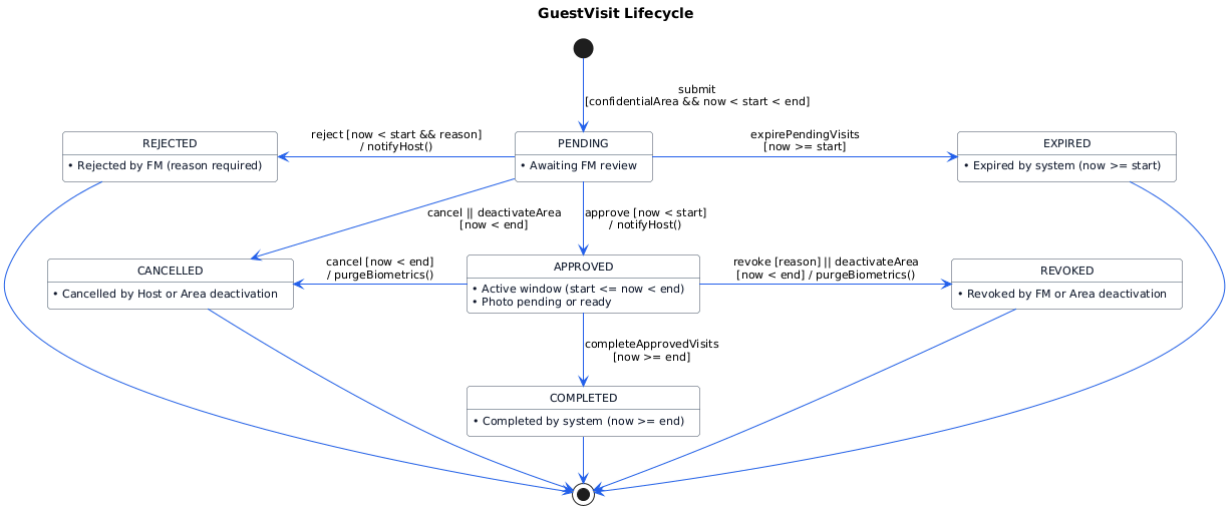
3.2 <Feature/Function Name2>

…

4. State Machine Diagram

4.1. GuestVisit

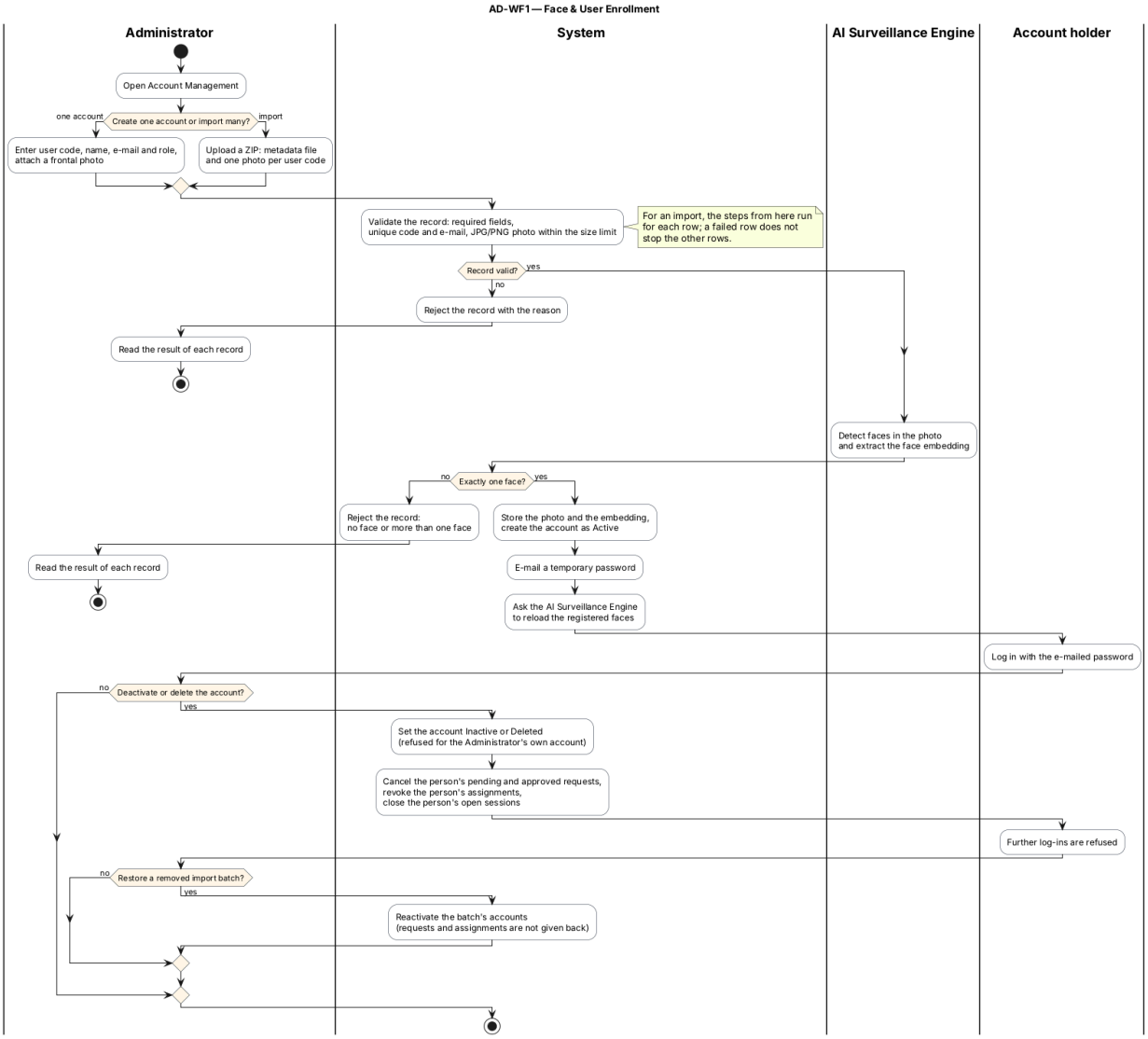
Link diagram: Ấn vào đây



5. Activity Diagram

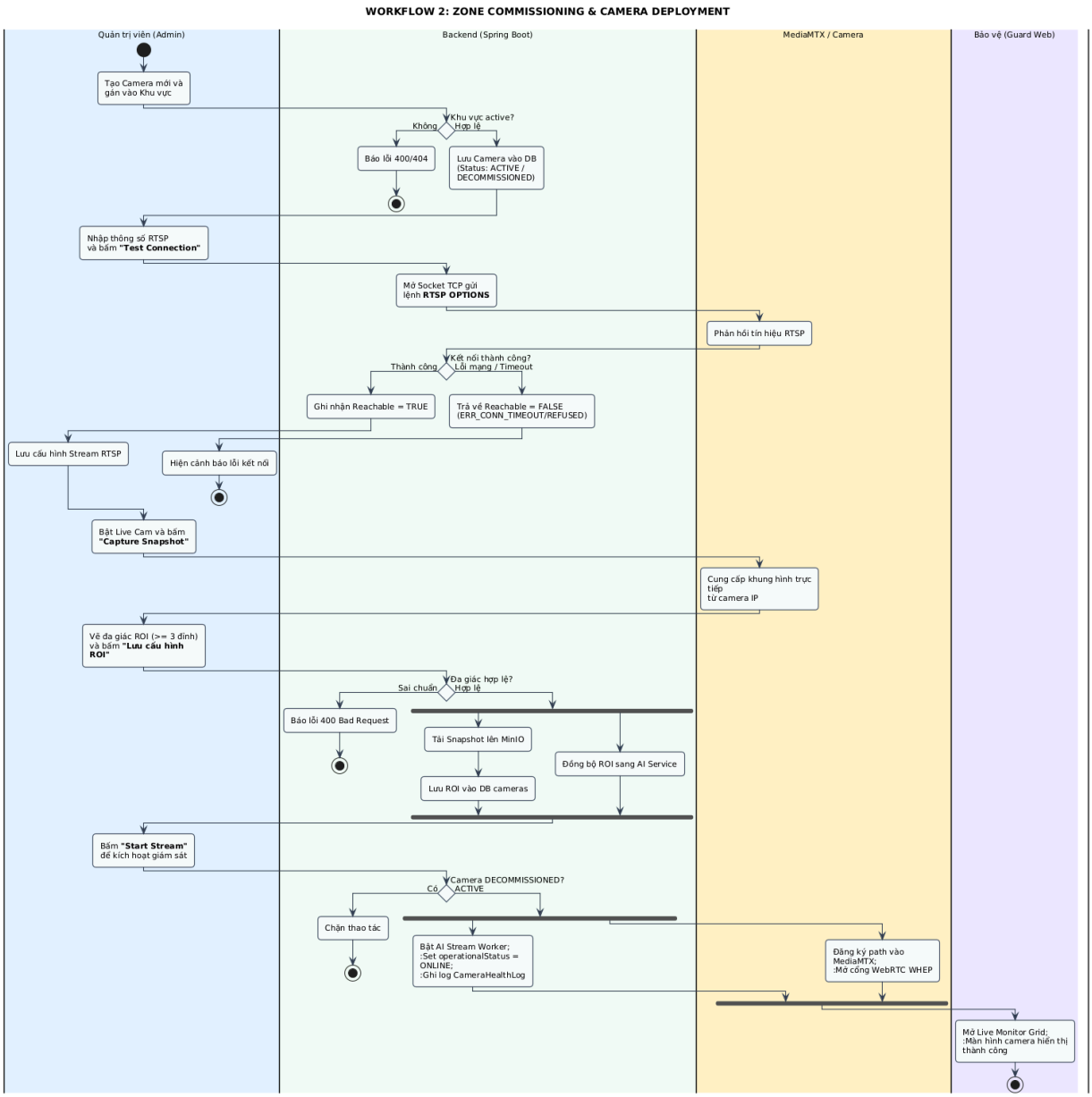
5.1. Face & User Enrollment

Link diagram: Ấn vào đây



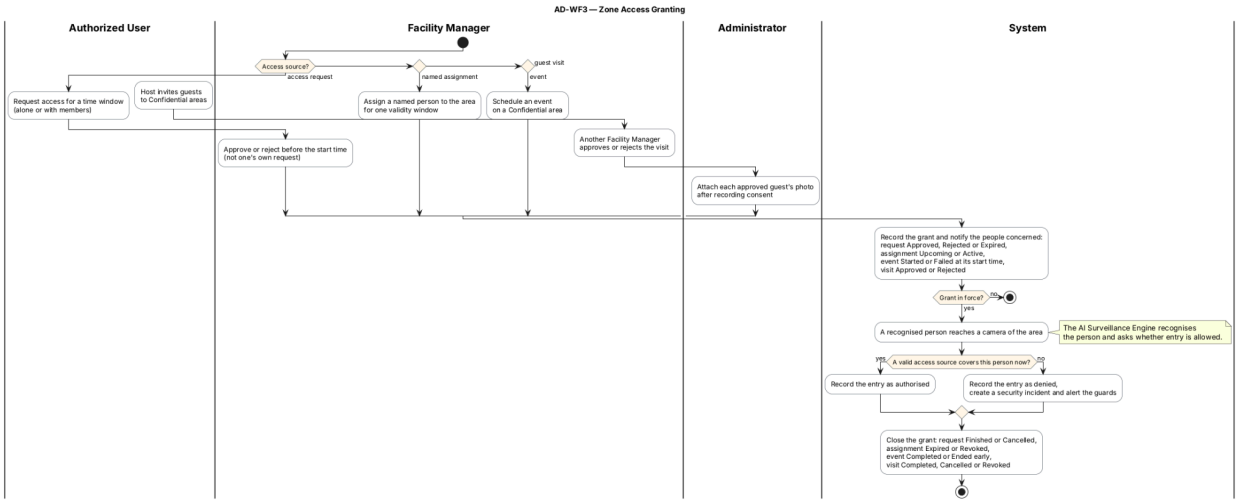
5.2. Zone Commissioning & Camera Surveillance Deployment

Link diagram: Ấn vào đây



5.3. Zone Access Granting

Link diagram: [Ấn vào đây]

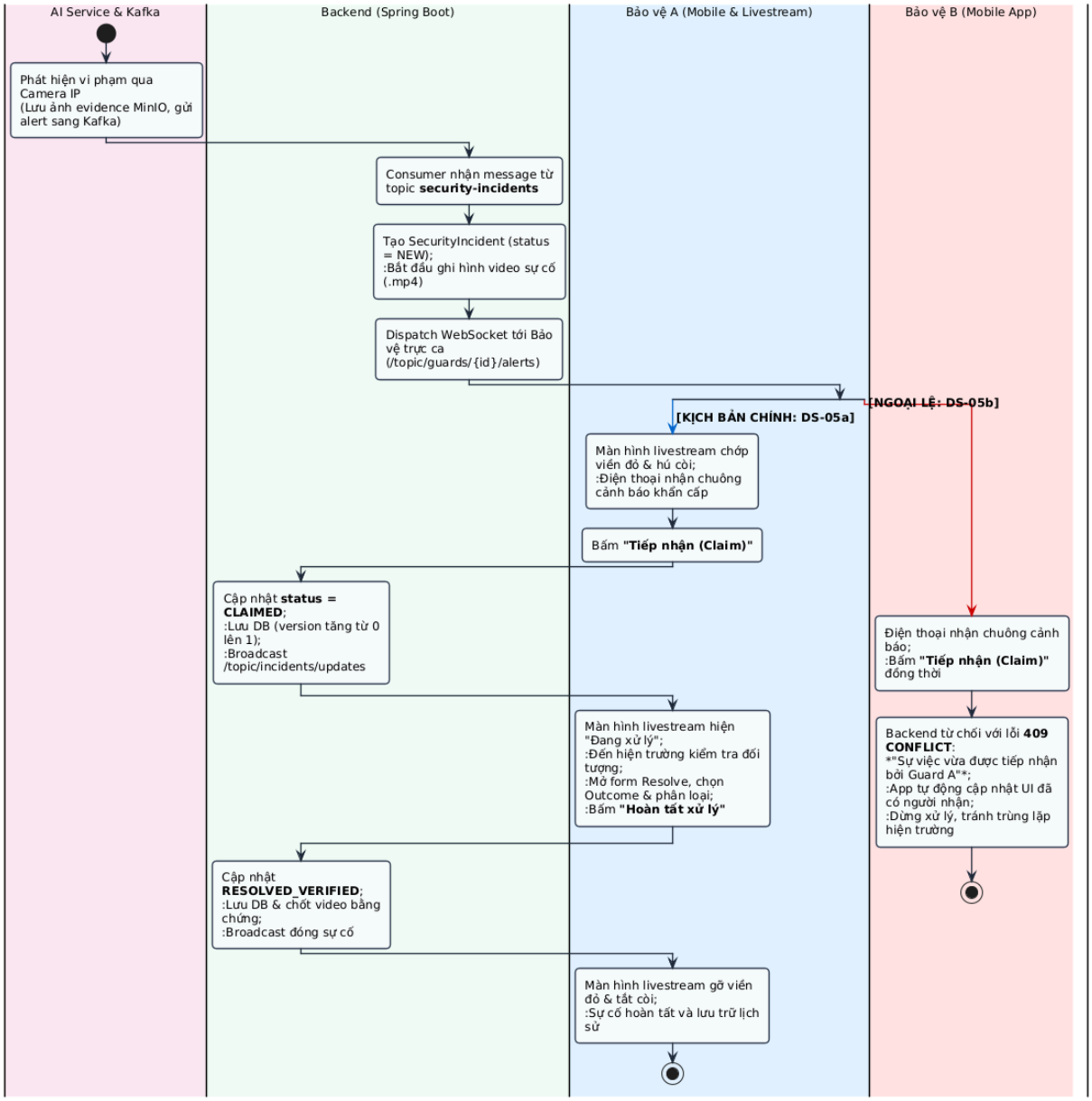


5.4. Real-time AI-powered Surveillance

5.5. Incident Alert Handling & Dispatch

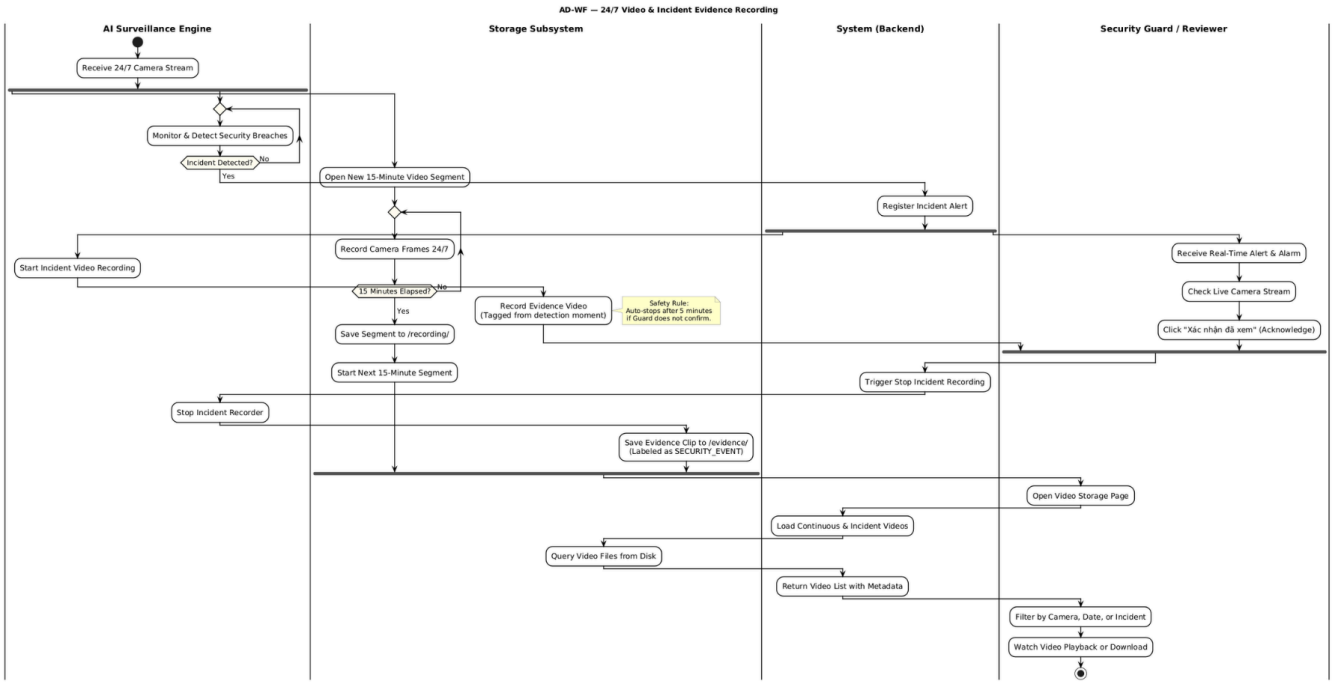
Link diagram: Ấn vào đây

WORKFLOW 5: INCIDENT ALERT HANDLING & DISPATCH



5.6. 24/7 Video & Incident Evidence Recording

Link diagram:



6. Technology Disclosure

Group	Category / Scope	Summary & Applied Technologies
Third-party services	*Service name, purpose (Authentication, Storage, Payment, AI APIs, etc.), data sent*	*• Google OAuth 2.0: Xác thực tài khoản người dùng và quản trị viên.* *• Cloud Object Storage (MinIO / AWS S3): Lưu trữ ảnh dataset khuôn mặt và snapshot bằng chứng.* *• Media Gateway (MediaMTX): Chuyển đổi luồng camera RTSP sang WebRTC/HLS độ trễ thấp.* *• Email Service (Google SMTP): Gửi thông báo cấp phát tài khoản và đặt lại mật khẩu.*
Development technologies	*Front-end, Back-end, Mobile, Database, with versions*	*• Frontend: React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, Zustand, MapLibre GL.* *• Backend Core: Java 21, Spring Boot 3, Spring Security, Spring Kafka, Spring WebSocket (STOMP).* *• AI Service: Python 3.11, FastAPI, PyTorch, OpenCV, ONNX Runtime.* *• Mobile: Flutter / Dart, Provider, Dio, Flutter Map, Background Task Service.* *• Database & Broker: PostgreSQL 16 (pgvector), Apache Kafka.*

Source code and DevOps	*Platform (GitHub, GitLab, Azure DevOps), CI/CD, branching rules*	*• VCS: GitHub (quản lý repository, Pull Request workflow).* *• CI/CD: GitHub Actions tự động kiểm thử backend (JUnit, PostgreSQL container) và linting/build frontend khi tạo PR.* *• Branching: Mô hình Git Flow (nhánh main bảo vệ nghiêm ngặt; phát triển qua các nhánh feat/*, fix/*).*
Deployment environments	*Dev, staging, production; deployment locations; deployment and rollback methods*	*• Environments: Dev (Local Docker), Staging và Production (Cloud PaaS & IaaS).* *• Deployment Platform: Frontend trên Vercel; Backend & AI/Database triển khai trên Railway hoặc AWS EC2 (Dockerized Containers).* *• Deployment & Rollback: Tự động hóa qua Container Images / Git triggers; rollback tức thì bằng Container tag hoặc redeploy phiên bản commit trước đó.*
AI tools	*Tools, models, and stages of use (specification, design, coding, testing, review)*	*• AI Assistance: Claude, Gemini, Antigravity Agent, GitHub Copilot (hỗ trợ phân tích tài liệu, thiết kế hệ thống, coding và sinh unit test).* *• Embedded Deep Learning Models: YOLO11 (phát hiện đối tượng/theo dõi hành vi), OpenCV YuNet (nhận diện vị trí khuôn mặt), ArcFace ResNet-50 (trích xuất đặc trưng vector 512D).*